So the problem is that there are three unknowns, thetha, M, X, and there r 2 equations, which r complex, and someone has run many points, for different values of t, and saved only the X and Y final equation values, they have not given us time t, thetha, M, X.

Every point on the csv will be telling us for which t value which values were generated, but we don't know that so there r so many unknowns, thetha, M, X, and each n every row contributes one t as a unknown. (rows r shuffled)(not mentioned that not shuffled, all belong to a range that's all is mentined)

If we consider one row, we have 2 equations, and 4 unknowns (thetha, M, X, t) so cant solve.

If we consider 2 rows, (thetha, m, x, t1, t2) 5 unknowns and 4 equations, cant solve.

If we consider around 1500 rows (obtained from a code)

```
import pandas as pd



df = pd.read_csv("xy_data.csv")

print("Number of rows:", len(df))
```

then we have 1503 unknowns (3 of those common params, and 1500 for the t) and we have 3000 equations (each row has 2 for x one, for y one) which is, more equations than unknows, so the problems can be solved.

If the equations was without any square, sine, exponential, etc, if it had been non linear then it would be easy we can solve using many methods like gaussian elimination, matric inv, etc. but now we have no direct algebraic technique to solve directly.

So we cant solve these perfectly, exact values we cant get, so what if we do estimates of the values and then check a loss function to how close it resembles the data, and then try to reduce any error.

But we don't have t values to match curve exactly, we cn just guess the thetha, M, X, values and generate whole curve by varying t from 6 to 60.

But we wont know that which point corresponds to which point, so we consider the whole curve, and check how close is curve to the 1500 points.

As rows might be unordered we cant compare point by point.

So for every point we find the closest point on the generated curve. well do this for all the 1500 points.

FIRST INTUITION

A straightforward approach would be:

For each observed point,

compare it with all 20,000 generated points,

compute every Euclidean distance,

choose the smallest one.

That requires

1500×20000=30,000,000

distance computations for just one evaluation of θ, M and X.

During optimization, this evaluation must be performed hundreds or even thousands of times.

Doing 30 million comparisons every time would be unnecessarily slow.

so we build a KD-Tree from the 20,000 generated points.

A KD-Tree is a spatial data structure designed for fast nearest-neighbor search.

Instead of checking every point, it intelligently eliminates large regions of the search space and quickly returns the nearest point.

This reduces the computation, while returning exactly the same nearest neighbor.

Once the nearest point has been found for every observed point, we compute the distance between them.

If the guessed parameters are poor,

many distances will be large,

therefore the loss will also be large.

If the guessed parameters are close to the true values,

almost every observed point lies very close to the generated curve,

therefore the loss becomes very small.

The loss function used in Stage 1 is the mean squared nearest-neighbor distance.

$$\text{Loss} = \frac{1}{N}\sum_{i=1}^{N} d_i^2$$

where d_i is the Euclidean distance from the i^th observed point to the nearest point on the generated curve.

this loss does not require knowing the original t values.

It only measures how well the entire generated curve fits the observed point cloud.

Stage 1 – Finding a good initial estimate

Now I convert the original problem into an optimization problem. Instead of solving the equations directly, my objective is to find the values of theta, M, and X that minimize the mean squared nearest-neighbor distance between the generated curve and the observed point cloud.

I can think of the loss function as a black box. I give it values of theta, M, and X, it generates the curve, builds a KD-Tree, finds the nearest neighbor for every observed point, computes the mean squared distance, and returns a single loss value.

The optimizer simply tries different parameter values and keeps the ones that produce a smaller loss.

A gradient-based optimizer is not suitable here because it assumes the loss changes smoothly. In my problem, even a small change in the parameters can suddenly change the nearest-neighbor assignments, causing the loss to jump. Gradient-based methods can also get stuck in local minima.

Instead, I use Differential Evolution, which is a population-based global optimization algorithm. Rather than working with a single solution, it maintains a population of candidate solutions. Initially, it generates many random combinations of theta, M, and X within their allowed ranges. Each candidate is evaluated using the loss function.

The algorithm then creates new candidate solutions by combining existing ones through mutation and crossover, and keeps whichever candidate produces the lower loss. Over many generations, the population gradually moves toward regions with smaller loss values.

For this dataset, this gives good initial estimates of theta, M, and X.

However, I do not stop here because this stage still relies on an approximation. The curve is represented by 20,000 sampled points instead of the true continuous curve, and the KD-Tree only identifies the nearest sampled point. It does not recover the exact continuous value of t for each observation.

As a result, even though the estimated parameters are already very close to the true values, they are not yet the final solution. The purpose of this stage is only to provide a good starting point. Once the generated curve is close enough to the observed data, I can estimate the hidden t value of every point much more accurately, which is exactly what Stage 2 does.

Stage 2 – Recovering the hidden t values

After Stage 1, we already have good estimates for theta, M, and X. They are not perfect, but they are close enough that the generated curve almost overlaps the actual data.

In the beginning, we could not estimate the t value of each point because our guessed curve was completely different from the real one. Any nearest point on that wrong curve would give a meaningless t value. That is why Stage 1 only focused on finding the global parameters.

Now the situation is different. Since the curve is already very close to the data, every observed point has a nearby point on the generated curve. During Stage 1, the KD-Tree already found the nearest sampled point for every observation. Because every sampled point was generated from a known t value, this also gives us a rough estimate of t.

However, this t value is only approximate because the curve was sampled at only 20,000 points. The true point usually lies somewhere between two sampled points.

To improve it, I keep theta, M, and X fixed and optimize only one variable, t. For each observed point, I search for the value of t that minimizes the squared distance between the point and the continuous curve. Since this is only a one-dimensional optimization problem, it is both fast and accurate.

Instead of searching the entire range of t every time, I start from the approximate t returned by the KD-Tree and search only within a small window around it. If the optimizer reaches the edge of that window, I expand the window and search again until the minimum is safely inside the interval or the valid t range is reached.

After Stage 2, I have accurate estimates for the t value corresponding to every observed point. This establishes a correspondence between the data points and the model. At this point, the only remaining unknowns are theta, M, and X, which makes the final optimization in Stage 3 much easier and more accurate.

After Stage 2, I now have accurate estimates of the t value corresponding to every observed point, while theta, M, and X remain fixed at the values obtained from Stage 1. These t values establish a correspondence between the observed data and the model. With this correspondence available, Stage 3 can refine theta, M, and X much more accurately using nonlinear least-squares optimization.

Stage 3 – Refining the global parameters

After Stage 2, I have estimated the t value for every observed point. At this stage, I keep these t values fixed and refine only theta, M, and X.

The main reason for Stage 3 is that Stage 1 only solved an approximate problem. It matched points to a sampled version of the curve using a KD-Tree, so the result was limited by the sampling resolution. After Stage 2, I have continuous estimates of the t values, so I can optimize against the actual curve instead of its sampled approximation.

For each observed point, I use its estimated t value to compute the predicted x and y coordinates from the model. I then compare these predicted coordinates with the observed

coordinates. The differences between them are called residuals. Since every point contributes one residual for x and one for y, I get a residual vector containing all points. The least-squares optimizer adjusts theta, M, and X to minimize the sum of the squared residuals, giving a much more accurate estimate of the global parameters.

However, once theta, M, and X change, the best t value for each point also changes slightly because the curve has moved. So, after Stage 3, I run Stage 2 again to update the t values, then run Stage 3 again to refine theta, M, and X. I repeat these two stages until the improvement becomes very small. This alternating process allows both the global parameters and the per-point t values to improve together until the solution converges.

CODE

```
import numpy as np
import pandas as pd
import scipy
from scipy.optimize import differential_evolution, minimize_scalar,
least_squares
from scipy.spatial import cKDTree
from functools import partial



# Curve model

def curve_xy(theta, M, X, t):
    ct, st = np.cos(theta), np.sin(theta)
    env = np.exp(M * np.abs(t)) * np.sin(0.3 * t)
    x = t * ct - env * st + X
    y = 42 + t * st + env * ct
    return x, y




# STAGE 1



def _stage1_loss(params, pts, t_grid):
    theta, M, X = params
    x, y = curve_xy(theta, M, X, t_grid)
    tree = cKDTree(np.column_stack([x, y]))
    d, _ = tree.query(pts, workers=-1)
    return np.mean(d ** 2)
```

```
def stage1(pts, t_grid, bounds_list, seed=42,
           maxiter=300, popsize=20, strategy="best1bin"):

    obj = partial(_stage1_loss, pts=pts, t_grid=t_grid)
    result = differential_evolution(
        obj, bounds=bounds_list, seed=seed,
        strategy=strategy,
        mutation=(0.5, 1.0), recombination=0.7,
        tol=1e-8, atol=1e-12,
        init='sobol',
        maxiter=maxiter, popsize=popsize, polish=True,
        updating='deferred', workers=-1
    )
    return result.x, result.fun



# STAGE 2:


def _point_sq_dist(t, ct, st, M, X, xi, yi):
    env = np.exp(M * np.abs(t)) * np.sin(0.3 * t)
    xt = t * ct - env * st + X
    yt = 42 + t * st + env * ct
    return (xt - xi) ** 2 + (yt - yi) ** 2

def stage2(theta, M, X, t_init, pts, t_grid, grid_spacing, T_MIN,
T_MAX, N):
    x_g, y_g = curve_xy(theta, M, X, t_grid)
    tree = cKDTree(np.column_stack([x_g, y_g]))
    if t_init is None:
        _, idx = tree.query(pts, workers=-1)
        t_init = t_grid[idx]

    ct, st = np.cos(theta), np.sin(theta)   # computed once, reused for
all N points
    t_out = np.empty(N)
    base_window = max(0.02, 5 * grid_spacing)
    edge_tol = 1e-12
    for i in range(N):
        xi, yi = pts[i]
        window = base_window
        for attempt in range(6):
            lo, hi = max(T_MIN, t_init[i] - window), min(T_MAX,
t_init[i] + window)
            res = minimize_scalar(_point_sq_dist, bounds=(lo, hi),
method='bounded',
```

```
                                    args=(ct, st, M, X, xi, yi),
options={'xatol': 1e-12})
            hit_boundary = np.isclose(res.x, lo, atol=1e-9) or
np.isclose(res.x, hi, atol=1e-9)

            at_domain_edge = (lo <= T_MIN + edge_tol) or (hi >= T_MAX -
edge_tol)
            if not hit_boundary or window >= (T_MAX - T_MIN) or
(hit_boundary and at_domain_edge):
                break
            window = min(window * 2, T_MAX - T_MIN)
        t_out[i] = res.x
    return t_out



# STAGE 3


def _residuals(params, t_arr, pts):
    th, m, x = params
    xt, yt = curve_xy(th, m, x, t_arr)
    return np.concatenate([xt - pts[:, 0], yt - pts[:, 1]])

def stage3(theta, M, X, t_arr, pts, bounds_list):
    lsq = least_squares(_residuals, x0=[theta, M, X], args=(t_arr,
pts),
                        bounds=([b[0] for b in bounds_list], [b[1] for
b in bounds_list]),
                        method='trf', xtol=1e-12, ftol=1e-12, gtol=1e-
12)
    return lsq.x, lsq.cost



# ALTERNATING BLOCK COORDINATE OPTIMIZATION


def alternating_fit(theta, M, X, pts, t_grid, grid_spacing, T_MIN,
T_MAX, N, bounds_list,
                    rel_cost_tol=1e-10, max_outer=40, verbose=False):
    t_curr = None
    prev_cost = None
    cost = None
    iterations_used = 0
    for it in range(max_outer):
```

```
        t_curr = stage2(theta, M, X, t_curr, pts, t_grid, grid_spacing,
T_MIN, T_MAX, N)
        (theta, M, X), cost = stage3(theta, M, X, t_curr, pts,
bounds_list)
        iterations_used = it + 1
        if prev_cost is not None:
            denom = max(prev_cost, cost, 1e-300)
            rel_delta = abs(prev_cost - cost) / denom
            if verbose:
                print(f"  iter {it}: cost={cost:.6e}
rel_delta={rel_delta:.3e}")
            if rel_delta < rel_cost_tol:
                break
        prev_cost = cost
    return theta, M, X, t_curr, iterations_used, cost



# REPORTING


def report_results(theta, M, X, t_curr, pts, stage1_loss=None,
final_cost=None, iterations_used=None):
    xt, yt = curve_xy(theta, M, X, t_curr)
    l1_per_point = np.abs(xt - pts[:, 0]) + np.abs(yt - pts[:, 1])

    print(f"NumPy {np.__version__}, SciPy {scipy.__version__}")
    if stage1_loss is not None:
        print(f"Stage 1 loss (grid-approx. mean sq. NN distance):
{stage1_loss:.6e}")
    if final_cost is not None:
        print(f"Final least-squares cost (0.5*sum(residuals**2)):
{final_cost:.6e}")
    if iterations_used is not None:
        print(f"Outer (Stage 2 <-> Stage 3) iterations used:
{iterations_used}")
    print(f"Mean L1 distance: {l1_per_point.mean():.6e}   Max:
{l1_per_point.max():.6e}")
    print(f"theta = {theta:.12f} rad = {np.degrees(theta):.12f} deg")
    print(f"M     = {M:.12f}")
    print(f"X     = {X:.12f}")

    desmos = (f"\\left(t*\\cos({theta:.12f})-
e^{{{M:.12f}\\left|t\\right|}}"
              f"\\cdot\\sin(0.3t)\\sin({theta:.12f})+{X:.12f},"

f"42+t*\\sin({theta:.12f})+e^{{{M:.12f}\\left|t\\right|}}"
              f"\\cdot\\sin(0.3t)\\cos({theta:.12f})\\right)")
```

```
    print(desmos)
    return desmos



# MAIN ENTRY POINT


def run_pipeline(csv_path, n_grid=20000, **stage1_kwargs):
    df = pd.read_csv(csv_path)
    pts = df[['x', 'y']].values
    N = len(pts)
    T_MIN, T_MAX = 6, 60

    bounds_list = [
        (np.radians(0.0001), np.radians(50)),  # theta
        (-0.05, 0.05),                          # M
        (0.0, 100.0),                           # X
    ]

    t_grid = np.linspace(T_MIN, T_MAX, n_grid)
    grid_spacing = t_grid[1] - t_grid[0]

    (theta, M, X), s1_loss = stage1(pts, t_grid, bounds_list,
**stage1_kwargs)
    theta, M, X, t_curr, iters, final_cost = alternating_fit(
        theta, M, X, pts, t_grid, grid_spacing, T_MIN, T_MAX, N,
bounds_list
    )
    return theta, M, X, t_curr, pts, s1_loss, final_cost, iters


if __name__ == "__main__":
    theta, M, X, t_curr, pts, s1_loss, final_cost, iters =
run_pipeline('xy_data.csv')
    report_results(theta, M, X, t_curr, pts, stage1_loss=s1_loss,
final_cost=final_cost, iterations_used=iters)
```

OUTPUT

```
NumPy 2.0.2, SciPy 1.16.3
Stage 1 loss (grid-approx. mean sq. NN distance): 8.632639e-07
Final least-squares cost (0.5*sum(residuals**2)): 5.492988e-09
```

```
Outer (Stage 2 <-> Stage 3) iterations used: 40
Mean L1 distance: 2.678099e-06   Max: 1.386784e-05
theta = 0.523598304142 rad = 29.999972987516 deg
M     = 0.029999997220
X     = 54.999998316480

\left(t*\cos(0.523598304142)-
e^{0.029999997220\left|t\right|}\cdot\sin(0.3t)\sin(0.523598304142)+54.
999998316480,42+t*\sin(0.523598304142)+e^{0.029999997220\left|t\right|}
\cdot\sin(0.3t)\cos(0.523598304142)\right)
```